A Simulation Tool for Exploring Organizational Learning from Software Incidents

David Pineda
Department of Computer Science
Brigham Young University
pined1@byu.edu

***Abstract*—Software teams can learn from incidents to prevent future failures, but how should organizations share this knowledge? We propose an agent-based simulation platform that, to the best of our knowledge, is among the first to operationalize absorptive capacity exploitation as measurable reliability outcomes—reduced incident rates, faster detection, and improved mitigation—enabling controlled comparison of knowledge-sharing strategies that practitioners currently choose without empirical guidance. Teams act as agents connected through organizational networks; learning follows a four-stage absorptive capacity model with explicit stage-transition probabilities and knowledge decay. We compare four sharing strategies (NONE, LOCAL, NEIGHBOR, GLOBAL) across varied organizational configurations, measuring reliability improvements against learning cost in developer-hours. Targeted ablation experiments and comparison against published incident ranges will provide evidence that simulation outputs are internally consistent and plausible.**

I. Introduction

Software systems fail. The question is whether teams learn from those failures to prevent them from happening again [1]. In modern distributed systems, this matters even more: one team's outage can cascade to other services, but one team's lessons can also help other teams avoid similar problems.

We want to understand how knowledge-sharing strategies affect reliability. Should teams only learn from their own incidents? Should they share with neighboring teams? Should everyone learn from every incident? These questions are hard to answer with real organizations. Researchers cannot randomly assign half a company to share knowledge globally while the other half shares nothing, then wait years to compare incident rates [3].

This work builds a general-purpose simulation platform to explore these questions. Unlike prior organizational simulations built for single studies [3], our platform is designed for reuse: researchers configure organizational structures, incident types, and learning rules without modifying code.

This work bridges organizational learning and software engineering—two fields that rarely inform each other. Organizational learning research has studied absorptive capacity—how organizations acquire, assimilate, and exploit external knowledge [5], [6]—but leaves "exploitation" abstract. Software engineering research has studied incidents and postmortems [2], [4], but lacks theoretical frameworks for how learning improves reliability. Empirical reviews find that sharing and storing lessons is the most underexposed subprocess in organizational learning from incidents—organizations consistently fail to move from lesson identification to organization-wide implementation [10]. **To the best of our knowledge, no prior simulation connects absorptive capacity exploitation to concrete reliability outcomes in software organizations. We address this gap by operationalizing exploitation as measurable improvements (reduced incident rates, faster detection, improved mitigation), enabling controlled comparison of knowledge-sharing strategies that are otherwise impossible to study in real organizations.**

This work has two goals:

- **Goal 1:** Build a configurable simulation platform that models teams learning from incidents through the four-stage absorptive capacity framework, with explicit stage-transition probabilities and knowledge decay.
- **Goal 2:** Use the platform to systematically compare four knowledge-sharing strategies (NONE, LOCAL, NEIGHBOR, GLOBAL), measuring reliability improvements against learning cost across varied organizational configurations.

Contributions:

- **A reusable simulation platform** separating configuration from logic, enabling systematic study of knowledge-sharing strategies without code modification.
- **A concrete operationalization of absorptive capacity** for incident learning: exploitation manifests as measurable prevention, detection, and mitigation improvements, bridging organizational theory and software engineering practice.

II. Background and Related Work

A. Organizational Learning and Absorptive Capacity

Cohen and Levinthal define absorptive capacity as the ability to recognize the value of new external information, assimilate it, and apply it [5]. Zahra and George reconceptualize this as four stages: acquisition, assimilation, transformation, and exploitation, distinguishing between potential absorptive capacity (PACAP: acquisition and assimilation) and realized absorptive capacity (RACAP: transformation and exploitation); social integration mechanisms determine the efficiency of conversion between them [6]. Our model implements all four stages to maintain theoretical fidelity. Argote et al.

frame organizational learning through three subprocesses—knowledge creation from experience, retention in repositories, and transfer between units [12]; our simulation operationalizes each: incidents produce knowledge increments (creation), δ governs depreciation (retention), and sharing strategies vary dissemination scope (transfer).

March identifies a fundamental trade-off between exploration and exploitation [7]. The exploitation bias—where immediate returns crowd out longer-horizon learning investment—explains why organizations systematically underinvest in incident postmortems even when doing so increases future reliability.

Applicability to software incident learning. The absorptive capacity framework was originally developed for R&D innovation. We argue it applies to software incident learning for three reasons. First, Reed's field study finds that engineers use postmortems to update mental models of complex systems—a direct analog to the assimilation and transformation stages [14]. Second, Drupsteen and Guldenmund identify that the hardest part of incident learning is translating understanding into action [10]—exactly the exploitation gap the framework targets. Third, the four stages map naturally onto postmortem practice: acquiring an incident report, understanding its root cause, connecting it to existing system knowledge, and implementing preventive changes [6], [16]. Whether software teams exhibit the same learning dynamics as R&D teams is an empirical question we do not resolve; we adopt the framework as a theoretically grounded starting point and treat its parameters as empirically tunable.

B. Learning from Software Incidents

Cook observes that complex systems always contain hidden failures [1]. Dekker argues errors are symptoms of system problems [9], leading to blameless postmortems now widely adopted at companies like Google [2]. Drupsteen and Guldenmund find that organizations most often fail at the transition from understanding to action [10]. Reed finds that high-performing organizations share rich context rather than action-item lists [14]. Dogga et al. analyze 2,000 Azure incidents, identifying root cause categories that inform our incident taxonomy [4]. Dingsøyr finds that only one in five software projects conduct postmortem reviews, and no organization in a 19-company study expressed satisfaction with their process [16]—underscoring the gap between incident learning potential and practice.

C. Simulation in Organizational Research

Harrison et al. argue simulation enables experiments impossible in real organizations [3]. Margaryan et al. identify simulation as an underused but appropriate method for learning-from-incidents research specifically, noting that multi-level organizational dynamics make controlled field experiments infeasible [13]. Sargent's validation framework distinguishes conceptual validity, model verification, and operational validity [17], organizing our evaluation strategy.

III. Methodology

A. Conceptual Overview

We build an agent-based simulator where each agent is a software team owning a subsystem in a distributed system. Teams connect through an organizational network; the sharing strategy determines which teams can learn from which incidents. Learning follows the absorptive capacity framework through four stages with explicit transition probabilities, and knowledge depreciates over time to model organizational forgetting.

B. Platform Architecture

We use agent-based modeling (ABM) because it naturally captures heterogeneous teams with local decision rules, emergent system-wide behavior, and network-mediated interactions—features difficult to represent in aggregate models like system dynamics [18]; Margaryan et al. identify simulation as an underused but appropriate tool for studying multi-level learning-from-incidents dynamics [13]. The platform separates configuration from simulation logic: researchers specify organizational structure, incident parameters, and learning rules through configuration files without modifying code. The simulation proceeds in discrete timesteps (one simulated day each), processing incident generation, learning propagation, and metric collection.

C. Inputs

Teams and Structure. Teams connect through a network (random, small-world [19], scale-free [20], or custom). Each team owns one subsystem per Conway's Law [11], [21] and maintains a knowledge vector with three dimensions (prevention, detection, mitigation), each in $[0, 1]$.

Incidents. Incidents are generated stochastically: each subsystem has a base incident rate modified by the team's prevention knowledge and recent deployment activity. Types follow Microsoft's ARTS taxonomy [4]; each subsystem has a vulnerability profile. Cascading failures across subsystems are out of scope.

Learning Scenarios. Four strategies: NONE (baseline), LOCAL (own incidents only), NEIGHBOR (own and adjacent team incidents), GLOBAL (all incidents organization-wide).

Deployments. Deployments occur stochastically at a configurable rate and temporarily elevate incident probability; prevention knowledge reduces this elevated risk.

D. Learning Process

Stage transition probabilities. Teams progress through four stages with configurable probabilities. Default values reflect the asymmetry between source teams and others: source teams acquire and assimilate automatically ($p = 1.0$, since they lived through the incident) but must invest effort in exploitation ($p_{\text{exploit}} = 0.7$). For other teams, stage probabilities are modulated by system similarity $s \in [0, 1]$, grounded in Nooteboom et al.'s cognitive distance theory [8], which predicts that absorptive capacity declines as knowledge distance between teams increases: $p_{\text{assimilate}} = 0.5 + 0.3s$, $p_{\text{transform}} =$

$0.4+0.3s$, $p_{\text{exploit}} = 0.3+0.3s$. These baseline coefficients are expert-estimated defaults, not empirically calibrated values; sensitivity analysis across plausible ranges identifies which assumptions most affect conclusions.

Knowledge update and decay. Consistent with Reed's finding that postmortems patch teams' mental models of system failure modes [14], knowledge increments ΔK_t represent updates to teams' understanding of prevention, detection, and mitigation: K_p reduces incident probability by $(1 - \alpha K_p)$; K_d reduces MTTD; K_m reduces severity and recovery time. Knowledge depreciates at rate δ per timestep to model personnel turnover and organizational forgetting: $K_t = K_{t-1}(1 - \delta) + \Delta K_t$. Following Darr et al., who document measurable knowledge depreciation across service organizations [22], we model decay as an inherent property of organizational memory; the specific default $\delta = 0.001$ per day (half-life $\approx$ 2 years) is a tunable baseline informed by typical software engineer tenure cycles, with sensitivity analysis exploring the range $[0.0005, 0.002]$.

Learning asymmetry. Source teams incur exploitation cost only; other teams must complete all four stages, with effort proportional to system dissimilarity. This asymmetry explains why cross-team learning is nontrivial and when broader sharing strategies justify their cognitive overhead.

Learning cost. Each completed stage consumes configurable developer-hours with estimated defaults: acquisition (0.5 hrs), assimilation (2 hrs), transformation (3 hrs), exploitation (4 hrs). These values represent conservative estimates of postmortem effort per stage and are treated as tunable parameters; robustness analysis varies them to confirm that cost-benefit conclusions hold across plausible ranges. Aggregate learning cost is tracked as a primary simulation output, enabling cost-benefit analysis across sharing strategies.

E. Outputs

Incident count, severity distribution, MTTD, availability (uptime percentage), and cumulative learning cost (developer-hours). The ratio of reliability improvement to learning cost across strategies is the primary outcome of interest.

IV. Evaluation

We follow Sargent's framework [17]: conceptual validity (are underlying theories appropriate?), computerized verification (is the code correct?), and operational validity (do outputs match expectations?). Unit tests and execution traces address verification; the following activities address conceptual and operational validity.

A. Verify Behavior

- **H1:** Broader sharing reduces total incidents: GLOBAL $<$ NEIGHBOR $<$ LOCAL $<$ NONE in mean incident count over 365 simulated days. Rejected if ordering fails in $>$20% of parameter configurations.
- **H2:** Deployment rate drives incidents: doubling deployment rate increases incident count by $\geq$20%, holding learning parameters constant.
- **H3:** In practice, full company-wide incident sharing is expensive—engineers get flooded with reports from teams they rarely interact with. A more realistic alternative is neighbor-based sharing: teams broadcast incidents only to adjacent teams in their communication network. The question is when this cheaper approach is "good enough." We hypothesize that there exist organizational conditions—specifically, team count and inter-team network connectivity—under which neighbor-based sharing captures at least 80% of the reliability benefit that company-wide sharing provides. We expect this threshold to depend on organizational scale: smaller, well-connected organizations may achieve near-equivalent outcomes through neighbor sharing alone, while larger or more sparsely connected organizations will show a meaningful reliability gap that justifies the overhead of global dissemination [8]. To test this, we will systematically vary team count and network topology (small-world, random, and scale-free structures [19], [20]) and measure the fraction of global sharing's reliability benefit that neighbor sharing recovers under each condition.
- **H4:** Network density accelerates knowledge spread: denser networks show faster mean team knowledge accumulation at simulation midpoint.

B. Ablation Experiments

To verify each component contributes meaningfully, we run targeted ablations:

- **No decay** ($\delta = 0$): verify reliability improvement is uniformly higher, confirming decay meaningfully affects outcomes.
- **No asymmetry:** set stage probabilities equal for source and other teams; verify this overestimates cross-team learning benefits.
- **No cost model:** remove developer-hour tracking; verify that the optimal sharing strategy changes when cost is ignored vs. included.

C. Test Robustness

We vary team count (6, 20, 50), network type (random, small-world, scale-free), deployment rate (low/medium/high), and learning effectiveness (weak/moderate/strong). We run 100+ simulations per configuration and report 95% confidence intervals, noting edge cases where neighbor sharing approaches global sharing's performance.

D. Partial Validation

We benchmark simulated MTTR against high-performing organizations ($<$1 hour per Forsgren et al. [23]) and incident type distributions against Dogga et al.'s Azure taxonomy [4]. Incident frequency and deployment rate baselines are treated as exploratory parameters rather than fixed calibration targets, as no published source provides per-team-per-year incident frequencies at the granularity our simulation requires. Deployment frequency and change failure rate ranges are drawn from large-scale empirical DevOps research [23], providing an

additional calibration anchor grounded in data from thousands of engineering organizations. We acknowledge this constitutes weak validation and does not substitute for calibration against real organizational data. Stronger validation—such as calibration against proprietary incident logs—is beyond the scope of this thesis but represents a natural direction for future work.

V. Discussion

A. Limitations

Synthetic data. Results reflect model predictions, not real organizational behavior.

Simplified model. Individual skill differences, office politics, and budget pressures are excluded for tractability. Static network topology does not capture real organizational restructuring. Cook observes that complex systems lack single root causes [1]; our use of Dogga et al.'s incident taxonomy is a necessary modeling simplification, not a claim that root causes are real.

Uncalibrated parameters. Findings are contingent on assumed parameter values. Sensitivity analysis identifies which assumptions most affect conclusions, but stronger validation would require proprietary incident logs.

Untested learning model. Whether software teams learn through absorptive capacity stages is empirical; future work could validate through practitioner interviews or longitudinal case studies.

Expert-estimated parameters. Stage transition probabilities and developer-hour cost defaults are expert estimates, not empirically calibrated values. While sensitivity analysis identifies which parameters most affect conclusions, the specific coefficients warrant future calibration against practitioner data.

Fixed psychological safety assumption. Following Edmondson [15] and Lunney and Lueder [2], our model assumes consistent blameless postmortem culture across all simulated organizations. Organizations with blame-oriented incident response would exhibit lower effective sharing rates; modeling variation in psychological safety is scoped as future work.

B. Practitioner Implications

While exploratory rather than predictive, simulation outputs suggest actionable hypotheses. If H3 holds—that organizational conditions exist under which neighbor-based sharing achieves $\geq$80% of company-wide sharing's reliability benefit—this suggests such organizations need not implement costly company-wide incident broadcasts. Instead, they could invest in strengthening inter-team connectivity and identifying meaningful team adjacency based on subsystem similarity. Results will be framed conditionally: "under model assumptions X, strategy Y yields Z% reliability improvement at C developer-hours cost per team per year," giving managers a structured framework for evaluating knowledge-sharing trade-offs rather than relying on intuition alone.

C. Significance

For researchers: The platform enables controlled experiments impossible in real organizations and provides a testable model connecting organizational learning theory to software reliability outcomes.

For practitioners: The platform makes knowledge-sharing trade-offs explicit and quantifiable: which strategy yields the most reliability per developer-hour invested?

Core questions this work can answer: How much reliability improvement justifies the overhead of global knowledge sharing? Under what conditions does neighbor-based sharing approximate global sharing's benefits at lower cost? When do diminishing returns make additional learning investment uneconomical? These questions require controlled experiments tractable only in simulation.

References

[1] R. I. Cook, "How complex systems fail," Cognitive Technologies Laboratory, Univ. Chicago, Chicago, IL, USA, Tech. Rep., 1998.

[2] J. Lunney and S. Lueder, "Postmortem culture: Learning from failure," in *Site Reliability Engineering*, B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds. O'Reilly Media, 2016, ch. 15.

[3] J. R. Harrison, Z. Lin, G. R. Carroll, and K. M. Carley, "Simulation modeling in organizational and management research," *Academy of Management Review*, vol. 32, no. 4, pp. 1229–1245, 2007.

[4] P. Dogga, C. Bansal, R. Costleigh, G. Jayagopal, S. Nath, and X. Zhang, "AutoARTS: Taxonomy, insights and tools for root cause labelling of incidents in Microsoft Azure," in *Proc. USENIX Annual Technical Conference*, 2023, pp. 359–372.

[5] W. M. Cohen and D. A. Levinthal, "Absorptive capacity: A new perspective on learning and innovation," *Administrative Science Quarterly*, vol. 35, no. 1, pp. 128–152, 1990.

[6] S. A. Zahra and G. George, "Absorptive capacity: A review, reconceptualization, and extension," *Academy of Management Review*, vol. 27, no. 2, pp. 185–203, 2002.

[7] J. G. March, "Exploration and exploitation in organizational learning," *Organization Science*, vol. 2, no. 1, pp. 71–87, 1991.

[8] B. Nooteboom, W. Van Haverbeke, G. Duysters, V. Gilsing, and A. van den Oord, "Optimal cognitive distance and absorptive capacity," *Research Policy*, vol. 36, no. 7, pp. 1016–1034, 2007.

[9] S. Dekker, *The Field Guide to Understanding 'Human Error'*, 3rd ed. Burlington, VT, USA: Ashgate Publishing, 2014.

[10] L. Drupsteen and F. W. Guldenmund, "What is learning from incidents, accidents and other unwanted events?" *Safety Science*, vol. 68, pp. 8–17, 2014.

[11] M. E. Conway, "How do committees invent?" *Datamation*, vol. 14, no. 4, pp. 28–31, 1968.

[12] L. Argote, S. Lee, and J. Park, "Organizational learning processes and outcomes: Major findings and future research directions," *Management Science*, vol. 67, no. 9, pp. 5399–5429, 2021.

[13] A. Margaryan, H. Littlejohn, and G. Stanton, "Research and development agenda for learning from incidents," *Safety Science*, vol. 99, pp. 54–65, 2017.

[14] J. P. Reed, "Beyond the 'fix-it' treadmill," *ACM Queue*, vol. 17, no. 1, pp. 27–46, 2019.

[15] A. C. Edmondson, "Psychological safety and learning behavior in work teams," *Administrative Science Quarterly*, vol. 44, no. 4, pp. 350–383, 1999.

[16] T. Dingsøyr, "Postmortem reviews: Purpose and approaches in software engineering," *Information and Software Technology*, vol. 47, no. 5, pp. 293–303, 2005.

[17] R. G. Sargent, "Verification and validation of simulation models: An advanced tutorial," in *Proc. Winter Simulation Conference*, 2020, pp. 16–29.

[18] E. Bonabeau, "Agent-based modeling: Methods and techniques for simulating human systems," *Proceedings of the National Academy of Sciences*, vol. 99, no. S3, pp. 7280–7287, 2002.

[19] D. J. Watts and S. H. Strogatz, "Collective dynamics of 'small-world' networks," *Nature*, vol. 393, pp. 440–442, 1998.

[20] A. L. Barabási and R. Albert, "Emergence of scaling in random networks," *Science*, vol. 286, no. 5439, pp. 509–512, 1999.

[21] A. MacCormack, J. Rusnak, and C. Y. Baldwin, "Exploring the duality between product and organizational architectures: A test of the mirroring hypothesis," *Management Science*, vol. 58, no. 8, pp. 1309–1324, 2012.
[22] E. D. Darr, L. Argote, and D. Epple, "The acquisition, transfer, and depreciation of knowledge in service organizations," *Management Science*, vol. 41, no. 11, pp. 1750–1762, 1995.
[23] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps*, Portland, OR, USA: IT Revolution Press, 2018.